第32小时

模拟试题Ⅱ（下午论文）

试题一　论软件架构风格

软件体系结构风格是描述某一特定应用领域中系统组织方式的惯用模式。体系结构风格定义一个系统家族，即一个体系结构定义一个词汇表和一组约束。词汇表中包含一些构件和连接件类型，而这组约束指出系统是如何将这些构件和连接件组合起来的。体系结构风格反映了领域中众多系统所共有的结构和语义特性，并指导如何将各个模块和子系统有效地组织成一个完整的系统。

请围绕“论软件架构风格”论题，依次从以下三个方面进行论述。

（1）概要叙述你参与分析和设计的软件系统开发项目以及你所担任的主要工作。

（2）软件系统开发中常用的软件架构风格有哪些?详细阐述每种风格的具体含义。

（3）详细说明你所参与分析和设计的软件系统是采用什么软件架构风格的，并分析采用该架构风格设计的原因。

解析：常见的、经典的架构风格有：

（1）数据流风格：包括批处理体系结构风格、管道-过滤器风格。

（2）管道-过滤器风格：包括主程序/子程序风格、面向对象风格、层次型风格（C/S 架构、B/S 架构）。

（3）以数据为中心的风格：包括仓库体系结构风格、黑板体系结构风格。

（4）虚拟机风格：包括解释器风格、规则系统风格。

（5）独立构件架构风格：进程通信风格、事件系统风格。

（6）C2 风格：C2 风格也被认为是层次风格的一种。

此外，一些现代的体系结构风格如微服务、SOA 等也可以写在论文中。

试题二　论企业应用系统的层次式架构风格

层次式架构是软件体系结构设计中最为常用的一种架构形式，它为软件系统提供了一种在结

构、行为和属性方面的高级抽象，其核心思想是将系统组成为一种层次结构，每一层为上层服务，并作为下层客户。在一些层次系统中，除了一些精心挑选的输出函数外，内部的层接口只对相邻的层可见。层次式架构风格的每一层最多只影响两层，同时只要给相邻层提供相同的接口，也允许每层用不同的方法实现，这种方式也为软件重用提供了强大的支持。

大部分的应用会分成表现层（或称为展示层）、中间层（或称为业务层）、数据访问层（或称为持久层）和数据层。

请围绕“企业应用系统的层次式架构风格”论题，依次从以下三个方面进行论述。

（1）概要叙述你参与管理和开发的企业应用系统建设项目以及你在其中所承担的主要工作。

（2）请结合项目实际情况，指出你参与开发的应用系统都有哪些层次以及每个层次的主要功能。

（3）请结合项目实际情况，说明你的项目是如何使用层次式架构进行架构设计的。

解析：

（1）表现层。表现层主要负责接收用户的请求，对用户的输入、输出进行检查与控制，处理客户端的一些动作，包括控制页面跳转等，并向用户呈现最终的结果信息。可以使用 MVC 模式来设计表现层。

（2）中间层。中间层负责实现系统的业务功能，主要包括业务逻辑层组件、业务逻辑层工作流、业务逻辑层实体和业务逻辑层框架四个方面。业务逻辑层组件分为接口和实现类两个部分，接口用于定义业务逻辑组件，定义业务逻辑组件必须实现的方法。通常按模块来设计业务逻辑组件，每个模块设计为一个业务逻辑组件，并且每个业务逻辑组件以多个 DAO 组件作为基础，从而实现对外提供系统的业务逻辑服务。业务逻辑层工作流能够实现在多个参与者之间按照某种预定义的规则传递文档、信息或任务的过程自动进行，从而实现某个预期的业务目标，或者促进此目标的实现。业务逻辑层实体提供对业务数据及相关功能的状态编程访问，业务逻辑层实体数据可以使用具有复杂架构的数据来构建，这种数据通常来自数据库中的多个相关表。业务逻辑层实体数据可以作为业务过程的部分 I/O 参数传递，业务逻辑层的实体是可序列化的，以保持它们的当前状态。业务逻辑层是实现系统功能的核心组件，采用容器的形式，便于系统功能的开发、代码重用和管理。

（3）持久层。持久层主要负责数据的持久化存储，主要负责将业务数据存储在文件、数据库等持久化存储介质中。持久层可以使用在线访问、Data Access Object、Data Transfer Object、离线数据模式、对象/关系映射（Object/Relation Mapping）5 种数据访问方式。

试题三　论面向服务的架构设计

在面向服务的架构（Service-Oriented Architecture，SOA）中，服务的概念有了延伸，泛指系统对外提供的功能集。例如，在一个大型企业内部，可能存在进销存、人事档案和财务等多个系统，在实施 SOA 后，每个系统用于提供相应的服务，财务系统作为资金运作的重要环节，也向整个企业信息化系统提供财务处理的服务，那么财务系统的开放接口可以看成是一个服务。

请围绕“面向服务的架构设计”论题，依次从以下三方面进行论述。

（1）概要叙述你参与分析和开发的软件系统开发项目以及你所担任的主要工作。

（2）说明面向服务架构的主要协议和规范、标准，详细阐述每种协议和规范、标准的具体内容。

（3）说明面向服务架构的设计原则，详细阐述每种设计原则的具体内容。

解析：

（1）UDDI 协议：统一描述、发现和集成协议。包含了服务描述与发现的标准规范，它使得商业实体能够彼此发现；定义它们怎样在 Internet 上互相作用，并在一个全球的注册体系架构中共享信息。

（2）Web 服务描述语言（Web Services Description Language，WSDL）：是一个用来描述 Web 服务和说明如何与 Web 服务通信的 XML 语言。

（3）SOAP 协议：SOAP 是在分散或分布式的环境中交换信息的简单的协议，是一个基于 XML 的协议。

（4）REST 规范：为了让不同的软件或者应用程序在任何网络环境下都可以进行信息的互相传递。微服务对外就是以 REST API 的形式暴露给调用者。RESTful 即 REST 形式的，是对遵循 REST 设计思想同时满足设计约束的一类架构设计或应用程序的统称，这一类都可称为 RESTful，可以理解为资源表述性状态转移。

（5）通信协议标准：SOA 服务用消息进行通信，该消息通常使用 XML Schema 来定义（也称作 XML Schema Definition，XSD）。

SOA 的设计原则主要有：

（1）无状态。以避免服务请求者依赖于服务提供者的状态。

（2）单一实例。以高内聚的实现方法，来避免功能冗余。

（3）明确定义的接口。服务的接口由 WSDL 定义，用于指明服务的公共接口与其内部专用实现之间的界线。

（4）自包含和模块化。服务封装了那些在业务上稳定、重复出现的活动和组件，实现服务的功能实体是完全独立自主的，独立进行部署、版本控制、自我管理和恢复。

（5）粗粒度。服务数量不应该太大，依靠消息交互而不是远程过程调用（RPC），通常消息量较大，但是服务之间的交互频度较低。

（6）服务之间的松耦合性。服务使用者看到的是服务的接口，其位置、实现技术和当前状态等对使用者是不可见的，服务私有数据对服务使用者是不可见的。

（7）重用能力。服务应该是可以复用的。

（8）互操作性、兼容和策略声明。为了确保服务规约的全面和明确，利用策略来定义可配置的互操作语义，来描述特定服务的期望、控制其行为。利用策略声明确保服务期望和语义兼容性方面的完整和明确。

试题四　论基于架构的软件设计方法及应用

基于架构的软件设计（Architecture-Based Software Design，ABSD）方法以构成软件架构的商

业、质量和功能需求等要素来驱动整个软件开发过程。ABSD 是一个自顶向下，递归细化的软件开发方法，它以软件系统功能的分解为基础，通过选择架构风格实现质量和商业需求，并强调在架构设计过程中使用软件架构模板。采用 ABSD 方法，设计活动可以从项目总体功能框架明确后就开始，因此该方法特别适用于开发一些不能预先决定所有需求的软件系统，如软件产品线系统或长生命周期系统等，也可为需求不能在短时间内明确的软件项目提供指导。

请围绕“基于架构的软件开发方法及应用”论题，依次从以下三个方面进行论述。

（1）概要叙述你参与开发的、采用 ABSD 方法的软件项目以及你在其中所承担的主要工作。

（2）结合项目实际，详细说明采用 ABSD 方法进行软件开发时，需要经历哪些开发阶段?每个阶段包括哪些主要活动?

（3）阐述你在软件开发的过程中都遇到了哪些实际问题及解决方法。

解析：

采用 ABSD 方法进行软件开发时，需要经历架构需求、架构设计、架构文档化、架构复审、架构实现和架构演化 6 个阶段：

（1）架构需求。架构需求阶段需要明确用户对目标软件系统在功能、行为、性能、设计约束等方面的期望。其主要活动包括需求获取、标识构件和架构需求评审。需求获取活动需要定义开发人员必须实现的软件功能，使得用户能够完成他们的任务，从而满足功能需求。与此同时，还要获得软件质量属性，满足一些非功能性需求。标识构件活动首先需要获得系统的基本结构，然后对基本结构进行分组，最后将基本结构进行打包成构件。架构需求评审活动组织一个由系统涉众（用户、系统分析师、架构师、设计实现人员等）组成的小组，对架构需求及相关构件进行审查。审查的主要内容包括所获取的需求是否真实反映了用户需求，构件合并是否合理等。

（2）架构设计。这个阶段是一个迭代过程，利用架构需求生成并调整架构决策。主要活动包括提出架构模型、将已标识的构件映射到架构中、分析构件之间的相互作用、产生系统架构和架构设计评审。

（3）架构文档化。主要活动是对架构设计进行分析与整理，生成架构规格说明书和测试架构需求的质量设计说明书。

（4）架构复审。在一个主版本的软件架构分析之后，需要安排一次由外部人员（客户代表和领域专家）参加的架构复审。架构复审需要评价架构是否能够满足需求，质量属性需求是否在架构中得以体现，层次是否清晰，构件划分是否合理等。从而标识潜在的风险，及早发现架构设计中的缺陷和错误。

（5）架构实现。主要是对架构进行实现的过程，主要活动包括架构分析与设计、构件实现、构件组装和系统测试。

（6）架构演化。这个阶段主要解决用户在系统开发过程中发生的需求变更问题。主要活动包括架构演化计划、构件变动、更新构件的相互作用、构件的组装与测试和技术评审。

在软件开发的过程中可能遇到的问题主要包括：在架构需求获取过程中如何对捕获的架构需求进行筛选和优先级排序；在架构复审过程中如何解决评审人员意见不一致的问题；在架构实现过程中如何根据项目组实际情况选择开发语言与开发平台；在架构演化过程中如何筛选并处理用户的需求变更等。